

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1703

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes

Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

*I **E.Ragul \ 192571032** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E. Ragul

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

Answer (a):

Dataset Loading and Pre-processing

The Iris dataset was loaded using scikit-learn's built-in loader. The dataset contains 150 samples with four numerical features (sepal length, sepal width, petal length, petal width) and one categorical target (species: setosa, versicolor, virginica). A check for missing values confirmed that the dataset is complete, with zero missing entries in every column, so no imputation was required. Since all four input features are already continuous numerical measurements, no categorical encoding was necessary for the inputs; the target species labels were kept as strings for readability during evaluation.

The data was split into training and testing sets using a 70:30 ratio with stratified sampling on the species column, ensuring that each class is proportionally represented in both sets. This produced 105 training samples and 45 testing samples.

First 5 Rows of the Dataset

Sepal L.	Sepal W.	Petal L.	Petal W.	Species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa

Data Types

Column	Data Type
sepal length (cm)	float64
sepal width (cm)	float64
petal length (cm)	float64
petal width (cm)	float64
species	object (string)

Train set shape: 105 samples x 4 features. Test set shape: 45 samples x 4 features.

(b)Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

Answer (b):

Model Building

A Decision Tree classifier was built using the Gini Index as the splitting criterion (criterion='gini'), with a maximum depth of 4 to keep the tree readable while retaining strong accuracy. The model was trained on the 105-sample training set described above.

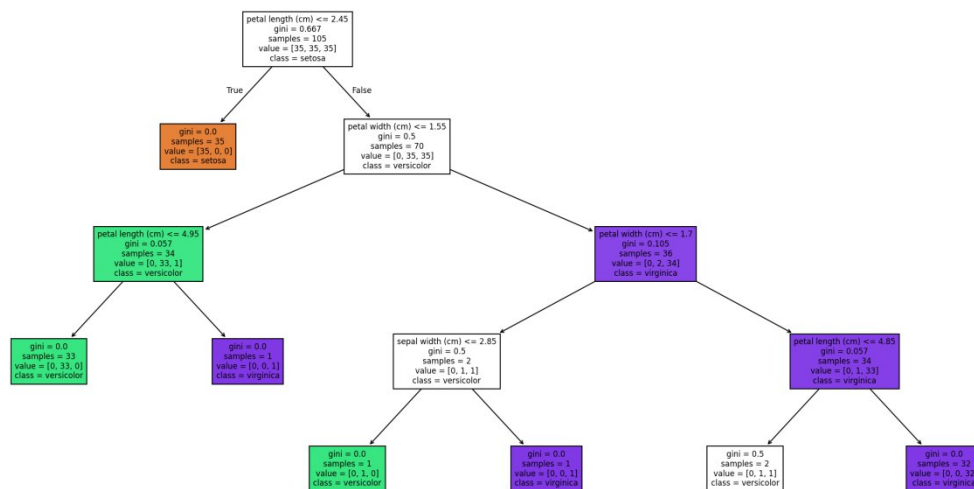
Tree Structure (text representation)

```

|--- petal length (cm) <= 2.45
|   |--- class: setosa
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.55
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: versicolor
|   |   |   |--- petal length (cm) > 4.95
|   |   |   |   |--- class: virginica
|   |   |--- petal width (cm) > 1.55
|   |   |   |--- petal width (cm) <= 1.70
|   |   |   |   |--- sepal width (cm) <= 2.85
|   |   |   |   |   |--- class: versicolor
|   |   |   |   |   |--- sepal width (cm) > 2.85
|   |   |   |   |   |   |--- class: virginica
|   |   |   |--- petal width (cm) > 1.70
|   |   |   |   |--- petal length (cm) <= 4.85
|   |   |   |   |   |--- class: versicolor
|   |   |   |   |   |--- petal length (cm) > 4.85
|   |   |   |   |   |   |--- class: virginica

```

Visualised Tree



Root Node and Justification

The root node splits on petal length (cm) ≤ 2.45 . This feature was selected by the algorithm because, among all four available features, splitting on this threshold produces the largest reduction in Gini impurity: it separates the setosa class from the other two species perfectly (Gini = 0.0 in the resulting left branch), which is the single most informative split available in the dataset. This matches the computed feature importance scores, where petal length (0.549) and petal width (0.436) together account for almost all of the model's decision-making, while sepal length contributes essentially nothing (importance = 0.0) and sepal width contributes very little (0.014).

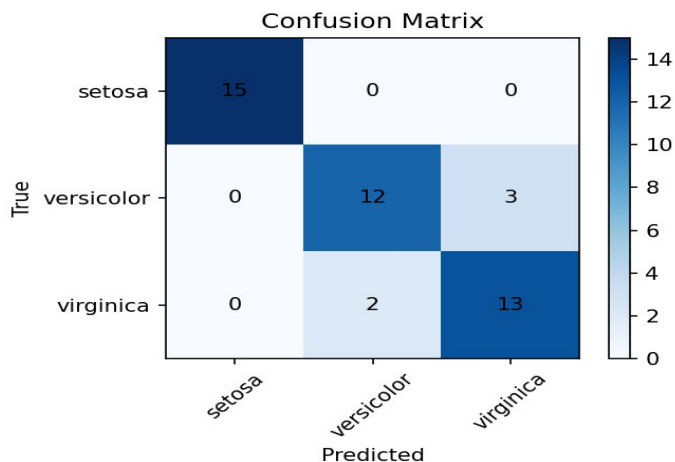
(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Answer (c):

Accuracy and Confusion Matrix

The trained model was evaluated on the held-out 45-sample test set, achieving an overall accuracy of 88.9% (40 out of 45 instances correctly classified).

True \ Predicted	setosa	versicolor	virginica
setosa	15	0	0
versicolor	0	12	3
virginica	0	2	13



Classification Report

Class	Precision	Recall	F1-score	Support
setosa	1.00	1.00	1.00	15
versicolor	0.86	0.80	0.83	15
virginica	0.81	0.87	0.84	15
Accuracy			0.89	45

Performance Commentary

The model classifies setosa perfectly, which is expected since setosa is linearly separable from the other two species based on petal measurements alone, as seen at the root split. All of the classification errors occur between versicolor and virginica, which is consistent with the known overlap in petal length and width between these two species in the Iris dataset. Overall, an accuracy of 88.9% with balanced precision and recall across classes indicates a reasonably strong model given its restricted depth of 4; allowing greater depth would likely resolve some of the borderline cases at the cost of a more complex, less interpretable tree.

Misclassified Instances

Sepal L.	Sepal W.	Petal L.	Petal W.	True	Predicted
6.3	3.3	4.7	1.6	versicolor	virginica
6.0	3.0	4.8	1.8	virginica	versicolor

Both misclassified instances lie very close to the petal width = 1.55-1.70 decision boundary used deeper in the tree, which explains why the model struggles with them: their measurements sit in the overlapping region between versicolor and virginica, where no single threshold on the available features can separate the classes perfectly.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).

Answer (a):

Environment Definition

The environment is a 4x4 Grid World containing 16 states, numbered 0 to 15 in row-major order (state = row*4 + column). The agent starts at state 0 (top-left corner). State 15 (bottom-right corner) is defined as the Goal, and state 10 is defined as an Obstacle. Four actions are available at every state: Up, Down, Left, and Right, each of which moves the agent one cell in the corresponding direction, with movement clipped at the grid boundary (an action that would move the agent off the grid leaves it in place).

Reward Structure

The agent receives a reward of +10 for reaching the Goal state (state 15), a reward of -5 for entering the Obstacle state (state 10), and a reward of -1 for every other step, which encourages the agent to reach the goal in as few steps as possible while avoiding the obstacle.

Q-table Initialisation and Hyperparameters

The Q-table was initialised as a 16 x 4 matrix of zeros, with one row per state and one column per action. The following hyperparameters were used for training:

Hyperparameter	Symbol	Value	Purpose
Learning rate	alpha	0.1	Controls how much new information

			overrides old Q-values
Discount factor	gamma	0.9	Weighs the importance of future rewards
Exploration rate	epsilon	0.2	Probability of choosing a random action (epsilon-greedy)

(b)Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

Answer (b):

Training Procedure

The Q-Learning algorithm was run for 100 episodes. In each episode, the agent starts at state 0 and follows an epsilon-greedy policy (choosing a random action with probability $\epsilon = 0.2$, otherwise the action with the highest current Q-value) until it reaches the Goal or a maximum of 100 steps is reached. After every action, the Q-table is updated using the standard Q-Learning update rule: $Q(s,a) = Q(s,a) + \alpha * [r + \gamma * \max(Q(s_next)) - Q(s,a)]$.

Q-value Evolution for Two Representative States

State 0 (start corner) and State 5 (an interior cell one step in from the top-left) were tracked across training. Their Q-values, in the order [Up, Down, Left, Right], evolved as follows:

Episode	State	Up	Down	Left	Right
1	0	-0.297	-0.288	-0.271	-0.199
1	5	-0.289	-0.280	-0.199	-0.199
50	0	-2.124	-2.049	-2.178	-2.054
50	5	-1.173	-0.938	-1.180	-0.248
100	0	-2.069	-2.106	-2.173	0.776
100	5	-1.173	-0.850	-1.180	2.028

At Episode 1, all Q-values are close to zero and slightly negative, reflecting the -1 step penalty accumulated during mostly random early exploration with no learned structure yet. By Episode 50, Q-values have become more negative overall (since the agent is still exploring paths that include detours and the obstacle penalty), but a preference for the Right action at State 5 is beginning to emerge (-0.248, clearly higher than the other three actions). By Episode 100, the Right action at both states has become clearly dominant and positive or near-positive (0.776 at State 0 and 2.028 at State 5), showing that the agent has learned that moving right and down towards the goal at state 15 is the most rewarding path, while the other actions remain negative, reflecting wasted or obstacle-adjacent moves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

Answer (c):

Final Learned Policy

Extracting the action with the highest Q-value at each state produces the following optimal policy, displayed as a 4x4 grid (R = Right, D = Down, X = Obstacle, G = Goal):

	Col 0	Col 1	Col 2	Col 3
Row 0	R	R	D	D
Row 1	R	R	R	D
Row 2	R	D	X (obstacle)	D
Row 3	R	R	R	G (goal)

The learned policy consistently routes the agent rightward and downward toward the goal at state 15 (row 3, column 3), and, importantly, every cell adjacent to the obstacle at state 10 (row 2, column 2) routes the agent around it rather than into it, confirming that the obstacle penalty was successfully learned and avoided.

Convergence Behaviour

The cumulative reward per episode starts extremely low (around -117 in Episode 1), since the agent initially wanders the grid with no learned guidance, frequently stepping into the obstacle and taking many steps before accidentally reaching the goal. Within the first 15-20 episodes, the cumulative reward rises sharply and then stabilises in a positive range (averaging around 3.6 over the final 10 episodes, compared to an average of -21.9 over the first 10 episodes), fluctuating slightly around a positive value because of the residual $\epsilon = 0.2$ exploration rate, which occasionally forces a random, sub-optimal action even after the optimal policy has been learned.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation